Technical Implementation Plan Ontology-Based Helpdesk App

Ontology-Based Helpdesk Application — Technical Implementation Plan (Transmodel / NeTEx / OpRa / SIRI / DATEX II / NAPCORE ITS)

1. Purpose and Scope

This document describes the technical design and implementation plan for an **ontology-driven Helpdesk and Knowledge Access application** for **Transmodel-based and ITS standards**, with an initial focus on **NeTEx**, **OpRa**, **SIRI**, and **DATEX II**, aligned under a **cross-standard Napcore ITS application ontology** (namespace `nits:`).

HOwThe system processes user questions, grounds them in a **federated, aligned ontology landscape** built from **locally cloned GitHub repositories**, applies **symbolic reasoning**, and produces **articulated, evidence-based answers** grounded in authoritative artifacts such as **XSD, XML, Markdown, and YAML**.

AI is used **only for controlled natural-language articulation**, never for deciding semantics, correctness, or interpretation of standards.

---

2. Architectural Decisions (Explicit)

The following decisions are **intentional and foundational**:

1. The ontology architecture is **federated**, not monolithic.
2. A **minimal Napcore ITS core ontology** (`nits:`) is used as the semantic entry point.
3. Standard-specific ontologies (NeTEx, OpRa, SIRI, DATEX II) remain **separate but aligned**.
4. Users **do not need to know which standards apply** — relevant standards are discovered through reasoning.
5. The ontology database is **GraphDB** (RDF/OWL-native).
6. Relational document storage is **PostgreSQL**.
7. Application backend is **Django (Python)**.
8. Frontend is **React + TypeScript + Vite**.
9. **Neo4j is explicitly excluded** from the initial architecture.

These decisions support correctness, explainability, performance, and governance.

---

3. High-Level System Architecture

Plain Text

```
User Question
    ↓
Parsing & Normalization
    ↓
Ontology Grounding (nits-core + aligned standard ontologies)
    ↓
Symbolic Reasoning (cross-standard if required)
    ↓
Answer Candidate + Evidence (XSD / XML / Spec references)
    ↓
AI Articulation (constrained, evidence-based NLG)
    ↓
Final User Answer

```

Key Principles

- **Federation over aggregation**: standards remain autonomous
- **Abstraction before specialization**
- **Artifact-grounded reasoning**
- **Deterministic logic before AI**
- **Explainability by design**

---

4. Knowledge Sources and Repository Management

4.1 Locally Cloned GitHub Repositories

All knowledge sources are **locally cloned GitHub repositories**, ensuring:

- Offline operation
- Full reproducibility
- Alignment to tagged releases or commits
- Standards governance compatibility

Initial repositories include:

- NeTEx
- OpRa
- SIRI
- DATEX II
- Shared Transmodel references

4.2 Repository Artifact Types

All artifact types are first-class inputs:

- **XSD** — structure, typing, cardinality, constraints

- **XML** — concrete usage examples
- **Markdown (MD)** — normative and explanatory text
- **YAML** — profiles, configuration, derived constraints

Artifacts are immutable; only **derived semantic representations** are created.

---

5. Ontology Landscape Design

5.1 Federated Ontology Model (Chosen)

The ontology landscape is **intentionally federated**:

- **Napcore ITS Core Ontology (`nits:`)**
  - Small, stable, fast-loading
  - Contains *only* cross-standard abstract concepts
  - Used for initial question grounding
- **Standard-Specific Ontologies**:
  - NeTEx ontology
  - OpRa ontology
  - SIRI ontology
  - DATEX II ontology
- **Alignment Ontologies**:
  - Explicit mappings between `nits:` and each standard ontology

No standard ontology is merged into another; **alignment replaces duplication**.

5.2 Why Not a Single Large Ontology

A single aggregated ontology was explicitly rejected because it:

- Blurs standard boundaries
- Increases reasoning complexity
- Makes governance and versioning difficult
- Encourages accidental concept duplication

Federation enables **cross-standard reasoning without semantic pollution**.

---

6. Concept Extraction and String Processing

A dedicated extraction layer supports:

- String-based identification of schema elements and types
- Detection of normative keywords (SHALL, SHOULD, MAY)

- Stable mapping from textual identifiers to ontology IRIs

Extraction combines:

- XML/XSD parsers
- Controlled string normalization
- SKOS-based labeling and synonym control

---

7. Reuse of Existing Ontologies

The system deliberately **reuses established vocabularies**:

- **SKOS** — terminology, labels, synonyms
- **Dublin Core / DCTERMS** — provenance and versioning
- **Schema.org** — generic concepts where appropriate
- **OWL / RDF / RDFS** — formal semantics

These ontologies are **imported**, not copied or redefined.

---

8. Ontology Storage and Reasoning Platform (Chosen)

8.1 GraphDB as Ontology Database

GraphDB is selected as the ontology and reasoning platform because it:

- Is RDF/OWL-native
- Supports named graphs
- Provides built-in reasoning profiles
- Offers SPARQL for transparent querying
- Is well-suited to large, aligned ontologies

Each standard ontology and alignment set is stored in **separate named graphs**, enabling scoped reasoning and traceability.

8.2 Explicit Exclusion of Neo4j

Neo4j is excluded from the initial design because:

- It lacks native OWL semantics
- It does not support standard reasoning profiles
- Alignment and inference would require custom logic

Neo4j may be introduced later **only for auxiliary analytics**, not semantic reasoning.

---

9. Application Architecture & Technology Stack

9.1 Backend

- **Django (Python)** — orchestration, APIs, security
- **GraphDB** — ontology storage and reasoning (via SPARQL)
- **PostgreSQL** — repository artifacts, metadata, questions, answers

Responsibilities are strictly separated:

- Django orchestrates
- PostgreSQL stores documents and metadata
- GraphDB stores meaning and logic

9.2 Frontend

- **React**
- **TypeScript**
- **Vite**

The frontend is responsible for:

- Question submission
- Evidence display
- Transparency ("why this answer")
- Editorial tooling (future)

---

10. Multi-Standard Reasoning (Key Capability)

When users do not specify standards:

1. Questions are grounded to `nits:` abstract concepts
2. Aligned standard concepts are discovered automatically
3. Relevant ontologies (e.g. NeTEx + OpRa + SIRI) are activated
4. Cross-standard reasoning is performed
5. Evidence is assembled per standard

This enables **correct answers even when the user is unaware of the underlying standards**.

---

11. AI Articulation Layer

The AI layer:

- Receives approved claims and evidence references
- Produces human-readable explanations
- Is constrained to **explain**, never infer or reinterpret

---

12. Editorial Curation & Learning Loop

All questions, groundings, and answers are persisted to support:

- Editorial review
- Ontology refinement
- Rule improvement
- Expansion to new standards

Learning happens in **ontologies and rules**, not in the AI model.

---

13. Non-Functional Requirements

- **Explainability**: file- and section-level traceability
- **Governance readiness**: versioned standards, ontologies, and alignments
- **Extensibility**: future ITS standards
- **Safety**: deterministic reasoning, constrained AI

---

14. Key Design Maxim

Artifacts define truth.

Ontologies align meaning.

Rules derive conclusions.

AI explains — never decides.

---

15. Parsing & Normalization Pipeline (Modern, Semantic Approach)

This section defines the **initial parsing and normalization pipeline** used to transform a raw user question into a canonical, ontology-grounded query object. Classical lexical analysis (POS tagging, stemming, syntactic parsing) is **explicitly not used**.

The pipeline is designed for **standards-grade correctness, explainability, and determinism**.

15.1 Pipeline Overview

Plain Text

```
Raw User Question
    ↓
Canonical Text Normalization
    ↓
Semantic Signal Extraction (intent, normativity)
    ↓
Domain Concept Extraction (schema- and ontology-driven)
    ↓
Abstract Anchoring (nits-core)
    ↓
Standard Scope Discovery (ontology alignments)
    ↓
Canonical Semantic Query Object

```

15.2 Step 1 – Canonical Text Normalization

Purpose: stabilize the input **without interpreting meaning**.

Applied operations:

- Unicode normalization (NFKC)
- Whitespace and punctuation normalization
- Preservation of case-sensitive identifiers (e.g. `ServiceJourneyPattern`)
- Preservation of acronyms (NeTEx, SIRI, DATEX II)

Not applied:

- stemming or lemmatization
- stopword removal
- full lowercasing

15.3 Step 2 – Semantic Signal Extraction

This step detects **high-level semantic signals**, not grammar.

Intent Detection (rule-first)

Typical intents in this domain:

- `normative_statusdefinition_locationcross_standard_relationcomparison`Detection is rule-based using stable linguistic patterns.

Normative Strength Detection (critical)

Modal keywords are detected explicitly:

- SHALL / MUST → mandatory
- SHOULD → recommended
- MAY → optional
- mandatory / optional → corresponding semantic strength

This information is later used directly in ontology reasoning.

15.4 Step 3 – Domain Concept Extraction (Controlled)

Concept extraction is performed against **known knowledge**, not open NER.

Matching sources:

- `skos:prefLabel` and `skos:altLabel` from ontologies
- XSD element and type identifiers
- Known standard names

Matching strategy:

- exact match
- CamelCase- and acronym-aware match
- confidence scoring

15.5 Step 4 – Abstract Anchoring in nits-core

Extracted domain terms are anchored to **abstract cross-standard concepts** in the `nits:` core ontology.

Example:

JSON

```json
{
  "original_term": "ServiceJourneyPattern",
  "anchored_core_concept": "nits:JourneyPattern"
}
```

This step deliberately avoids choosing standards.

15.6 Step 5 – Standard Scope Discovery (Ontology-Driven)

Using alignment axioms, the system discovers which standards are relevant.

Example alignments:

- `nits:JourneyPattern` ↔ `netex:ServiceJourneyPattern`
- `nits:JourneyPattern` ↔ `siri:JourneyPatternRef`

Standards are activated **only after this step**.

15.7 Step 6 – Canonical Semantic Query Object

The output of parsing & normalization is a structured object:

```json
{
  "intent": "normative_status",
  "normativity": "mandatory_or_optional",
  "core_concept": "nits:JourneyPattern",
  "candidate_standards": ["NeTEx", "SIRI"],
  "original_terms": ["ServiceJourneyPattern"],
  "confidence": {
    "intent": 0.9,
    "concept": 0.95
  }
}
```

Only this object proceeds to reasoning.

---

16. Django Service Pseudo-Code

```python
class QuestionParsingService:
    def parse(self, text: str) -> SemanticQuery:
        text = normalize_text(text)
        intent = detect_intent(text)
        normativity = detect_normativity(text)
        terms = extract_domain_terms(text)
        core_concept = anchor_to_nits(terms)
        standards = discover_standards(core_concept)
        return SemanticQuery(
            intent=intent,
            normativity=normativity,
            core_concept=core_concept,
            standards=standards,
            original_terms=terms
        )
```

---

17. GraphDB SPARQL Mapping (Examples)

17.1 Anchor Term to Core Concept

```sparql
SELECT ?core WHERE {
  ?standardConcept skos:prefLabel "ServiceJourneyPattern" .
  ?standardConcept owl:equivalentClass|rdfs:subClassOf ?core .
  FILTER(STRSTARTS(STR(?core), "http://napcore.eu/nits"))
}
```

17.2 Discover Relevant Standards

```sparql
SELECT DISTINCT ?standard WHERE {
  ?core owl:equivalentClass|rdfs:subClassOf ?standardConcept .
  ?standardConcept dc:source ?standard .
}
```

---

18. Failure Modes & Fallback Strategies

18.1 No Concept Match

Strategy:

- Ask clarification
- Offer closest known concepts
- Log for editorial review

18.2 Ambiguous Core Concept

Strategy:

- Return disambiguation question
- Reduce reasoning scope

18.3 Conflicting Standards

Strategy:

- Report conflicts explicitly
- Present per-standard conclusions with evidence

18.4 Partial Evidence Only

Strategy:

- Answer with confidence level
- Mark answer as provisional
- Flag for editorial enrichment

18.5 AI Safety Fallback

If evidence is insufficient:

- AI must respond with uncertainty
- No inferential expansion is allowed

---

16. Complete Tooling Description (Knowledge Engineering–Based)

This section describes the **complete tooling stack** supporting the Helpdesk system, viewed from a **knowledge engineering perspective** rather than a traditional NLP or chatbot architecture. Each tool is chosen to support determinism, explainability, and standards governance.

16.1 Overall Tooling Layers

The system tooling is organized into five logical layers:

Plain Text

```
User Interface Layer
↓
Semantic Preprocessing Layer
↓
Ontology & Reasoning Layer
↓
Evidence & Repository Layer
↓
Application Orchestration Layer

```

Each layer has a clearly bounded responsibility and avoids overlap with adjacent layers.

16.2 User Interface Layer

Tools:

- React
- TypeScript
- Vite

Responsibilities:

- Capture raw user questions
- Present structured answers and cited evidence
- Provide transparency ("why this answer")
- Support editorial and review workflows (future)

No semantic interpretation or preprocessing is performed in the frontend.

16.3 Semantic Preprocessing Layer (Non-NLP)

This layer replaces classical NLP pipelines with **controlled semantic preprocessing**.

Tools:

- Python standard library (`unicodedata`, `regex`)
- Rule configuration (YAML / JSON)

Responsibilities:

- Canonical text normalization (Unicode, punctuation, whitespace)
- Extraction of semantic signals (intent, normativity)
- Deterministic handling of modal language (SHALL / SHOULD / MAY)

Explicitly excluded tools:

- spaCy
- NLTK
- lemmatizers and stemmers
- transformer-based NER

16.4 Domain Concept Anchoring Tooling

Tools:

- GraphDB (queried via SPARQL)
- Optional in-memory label cache for performance
- SKOS labels (`skos:prefLabel`, `skos:altLabel`)

Responsibilities:

- Match user terms against known ontology labels
- Map textual identifiers to existing IRIs
- Prevent creation of unknown or speculative entities

Concept extraction is **knowledge-driven**, not model-driven.

16.5 Ontology & Reasoning Layer (Core)

Primary Tool:

- GraphDB

Why GraphDB:

- Native RDF / OWL support
- Built-in reasoning profiles
- Named graph support (essential for federated ontologies)
- SPARQL for transparent, explainable queries

Stored content:

- `nits-core` ontology (minimal abstractions)
- Standard-specific ontologies (NeTEx, OpRa, SIRI, DATEX II)
- Alignment ontologies
- XSD-derived axioms and constraints
- Versioned named graphs per standard

This layer performs all semantic inference and cross-standard reasoning.

16.6 Evidence & Repository Tooling

Tools:

- PostgreSQL
- Locally cloned Git repositories

Stored assets:

- XSD files
- XML examples
- Markdown specifications
- YAML profiles and constraints
- Metadata (version, commit, provenance)

This layer ensures that every answer is **traceable to authoritative artifacts**.

16.7 Application & Orchestration Layer

Tools:

- Django (Python)

Responsibilities:

- Orchestrate the full pipeline
- Manage calls to GraphDB via SPARQL
- Persist questions, queries, answers, and evidence
- Enforce safety constraints and scope limits
- Serve APIs to the frontend

Key artifact:

- Canonical Semantic Query Object (shared contract between parsing and reasoning)

16.8 AI Tooling (Constrained, Non-Authoritative)

Tools:

- LLM accessed via strict prompt contract

Role:

- Transform approved claims and evidence into readable explanations

Constraints:

- No new concepts
- No new rules
- No reinterpretation of standards
- Mandatory citation of evidence

AI tooling is explicitly **non-authoritative and replaceable**.

16.9 Failure Handling & Editorial Support

Tooling support:

- Django workflows
- PostgreSQL persistence
- GraphDB provenance and named graphs

Handled failure modes:

- Concept not found → clarification request
- Ambiguity → disambiguation prompt
- Cross-standard conflict → parallel evidence-based answers
- Partial evidence → provisional answer with confidence

No failure mode falls back to AI guessing.

16.10 Tooling Summary

	Layer	Primary Tools
1	User Interface	React, TypeScript, Vite
2	Semantic Preprocessing	Python stdlib, rule engine
3	Concept Anchoring	GraphDB, SPARQL, SKOS
4	Ontology & Reasoning	GraphDB
5	Evidence Storage	PostgreSQL, Git
6	Orchestration	Django

7	Language Generation	Constrained AI

End of document.